

AI Agent Engineering

Core Concepts & Architectural Principles

Building AI agents requires transitioning from a linear model interaction paradigm (prompt -> response) to a dynamic closed-loop execution pattern (sense -> plan -> act). This handbook outlines the critical knowledge, patterns, and design patterns required to build reliable, production-ready AI agents.

1. Core Agentic Loop: Sense-Plan-Act

Unlike standard pipelines, an agent operates within an execution loop:

- Sense (Input): The agent perceives the environment through user input or callback events.
- Plan (Reasoning): The LLM analyzes the inputs, sets goals, decomposes tasks, and selects tools.
- Act (Execution): The system runs the chosen tools (databases, external APIs, code execution) and feeds the results back into the system to restart the loop.

Key architecture styles include:

1. ReAct (Reasoning + Acting): Alternating between 'Thought' (explaining what to do), 'Action' (specifying tool to run), and 'Observation' (result from the tool). This is the foundation of tool use.
2. Plan-and-Execute: Decomposing a complex user request into a step-by-step list of tasks, then executing each sequentially, checking progress and replanning if a task fails.
3. State Machines (Cyclic Graphs): Mapping agent flows as state nodes and transitions (conditional edges). This allows building loops (e.g., code writer node -> tester node -> loop back if test fails) and is the standard for robust agentic systems (e.g., LangGraph).

2. Function Calling & Tool Execution

Tools allow LLMs to interact with the outside world. The framework flow is as follows:

- Schemas: You define tools using structured schemas (typically JSON Schema) detailing descriptions, argument names, and types. Excellent, clear descriptions are critical; the model uses them to decide when and how to call the tool.
- Model Output: The model outputs a structured payload (often a tool call containing tool name and arguments) instead of conversational text.
- System Interception: The application intercepts this output, executes the function locally, retrieves the outcome, formats it as a tool response message, and feeds it back to the LLM.

Key Engineering Guardrails:

- Validation: Always validate JSON outputs and argument types before executing code to prevent system failures.
- Refusal & Recovery: If the LLM generates bad tool parameters, catch the exception and pass the error message back to the LLM as the observation, prompting it to self-correct.

3. Memory Systems for Agents

Memory is what allows agents to build context and personalizations across time. It is split into:

- Short-Term Memory: The chat history. Contains system prompts, previous user messages, tool calls, and tool observations. Because of context window limits, short-term memory is often compressed using sliding windows (discarding oldest messages) or summarization (LLM summarizing historical turns).
- Long-Term Memory: Stored external knowledge retrieved on demand. Powered by embedding vectors and Vector

Databases (ChromaDB, Pinecone). The agent embeds the query, searches the DB, and injects the most similar documents as context (Retrieval-Augmented Generation or RAG).

- Episodic Memory: Recording step-by-step agent logs, decisions, and outcomes so the agent can learn from past trials (e.g., 'Last time I ran this tool it failed with Error X, so I will try Tool Y').

4. Multi-Agent Topologies

Complex tasks are best solved by specialized agents working together. Main design patterns include:

- Router Pattern: A single supervisor agent accepts user input and routes the task to one specialized worker agent (e.g., routing a bug fix to the coding agent vs a documentation fix to the docs agent).
- Supervisor/Orchestrator-Workers: An orchestrator coordinates a set of workers, assigns sub-tasks, gathers results, and synthesizes the final response. Perfect for workflows with step dependencies.
- Joint Collaboration (Mesh): Agents converse with each other in a shared room or thread, criticizing and refining the outputs (e.g., Code Generator -> Code Reviewer -> Security Evaluator -> Generator).

5. Production Best Practices & Guardrails

- Observability: Always log tracing data (e.g., using OpenTelemetry, LangSmith, or Langfuse). You must be able to trace every single turn, prompt template, tool input, output, and token cost.
- Guardrails: Implement strict input and output validation. Validate that user prompts are safe (no injections) and tool executions are restricted (e.g. read-only database connections, sandboxed file execution).
- Deterministic Fallbacks: When the agent hits maximum loop steps or is unable to solve a task, fall back gracefully to asking for human assistance instead of looping infinitely.